

CS 240 - Lab 5

TAs: Yash & Abhi

Spring 2025

Welcome to the fifth graded lab of this course!!

Instructions:

- This lab has 3 questions. The first Question is the graded inlab. Rest of the questions are ungraded.
- Create a folder on your Desktop with the following directory structure. The folder must contain only the two specified files:
`submission_<roll_number>/`
`|-- q1/`
`|-- classifiers.py`
- Once you have completed the work, open the terminal, type the following command, and then call a TA: `check-cs240`
- The TA will run `submit-cs240` to submit your work to the server. **It is your responsibility to ensure your work is submitted.** If you fail to inform us to submit your work, you will not be graded.

Q1. A Generative Start

In this lab, we will see how we can get the best of both generative and discriminative models. While discriminative models learn the posterior $P(Y|X)$ directly, generative models learn $P(X|Y = c)$ or even the joint distribution $P(X, y)$.

In the previous outlab, we saw the connection between LDA and logistic regression and that LDA takes a logistic regression like form, therefore implying that there exists a combination of logistic regression weights that are equal to the LDA derived weights.

Gaussian Discriminant Analysis (GDA)

Let us consider a binary classification problem where $y \in \{0, 1\}$. In GDA, we model the class-conditional densities as multivariate Gaussians:

$$p(x | y = k) = \mathcal{N}(x | \mu_k, \Sigma_k)$$

where μ_k is the mean vector and Σ_k is the covariance matrix for class k . The posterior probability is given by Bayes' rule:

$$p(y = 1 | x) = \frac{p(x | y = 1)\pi_1}{p(x | y = 1)\pi_1 + p(x | y = 0)\pi_0}$$

where $\pi_k = p(y = k)$ represents the class prior.

The Linear Assumption (LDA)

The general GDA model leads to a Quadratic Decision Boundary (QDA) because of the term $x^T \Sigma_k^{-1} x$. However, if we make the *Homoscedasticity Assumption* that both classes share the same covariance matrix Σ the quadratic terms cancel out.

Let $\Sigma_0 = \Sigma_1 = \Sigma$. The log-odds ratio becomes:

$$\begin{aligned} \log \frac{p(y = 1 | x)}{p(y = 0 | x)} &= \log \frac{\pi_1}{\pi_0} + \log \frac{\mathcal{N}(x | \mu_1, \Sigma)}{\mathcal{N}(x | \mu_0, \Sigma)} \\ &= \log \frac{\pi_1}{\pi_0} - \frac{1}{2}(x - \mu_1)^T \Sigma^{-1}(x - \mu_1) + \frac{1}{2}(x - \mu_0)^T \Sigma^{-1}(x - \mu_0) \end{aligned}$$

Expanding the quadratic terms, the $x^T \Sigma^{-1} x$ components cancel, leaving us with a linear function of x :

$$\log \frac{p(y = 1 | x)}{p(y = 0 | x)} = \mathbf{w}_{gen}^T x + b_{gen} \quad (1)$$

This confirms that Linear Discriminant Analysis (LDA) produces a linear decision boundary.

Recovering the Weights

By matching coefficients in the expansion of the equation above, we can derive the closed-form solution for the weights $\mathbf{w}_{gen}$ and bias b_{gen} in terms of the Gaussian parameters:

1. The Weights ($\mathbf{w}_{gen}$):

$$\mathbf{w}_{gen} = \Sigma^{-1}(\mu_1 - \mu_0)$$

2. The Bias (b_{gen}):

$$b_{gen} = -\frac{1}{2}\mu_1^T \Sigma^{-1} \mu_1 + \frac{1}{2}\mu_0^T \Sigma^{-1} \mu_0 + \log \left(\frac{\pi_1}{\pi_0} \right)$$

The Logistic Connection and Warm-Starting

Notice that Equation 1 implies that the posterior distribution takes the form of a sigmoid function:

$$p(y = 1 | x) = \sigma(\mathbf{w}_{gen}^T x + b_{gen}) = \frac{1}{1 + e^{-(\mathbf{w}_{gen}^T x + b_{gen})}}$$

This is **exactly** the hypothesis function used in Logistic Regression.

Why not just use LDA? While LDA and Logistic Regression share the same functional form, they optimize different objectives:

- **LDA** maximizes the *Joint Likelihood* $\prod p(x, y)$. It is data-efficient but biased if the Gaussian assumption is violated.
- **Logistic Regression** maximizes the *Conditional Likelihood* $\prod p(y | x)$. It is more robust to non-Gaussian data but requires iterative optimization (Gradient Descent).

You would by now notice that in general in any ML model, the more the bias (typically called inductive bias as it is deliberate) the lesser the data samples required. Inductive biases are generally all assumptionss that have been made about the data distribution, more formally $P(X, y)$ while designing your ML model.

The Warm Start Strategy In this lab, we will combine the strengths of both. Instead of initializing the Logistic Regression weights randomly (which may start far from the optimal solution), we will use the LDA solution $(\mathbf{w}_{gen}, b_{gen})$ as the initial point for Gradient Descent.

- **Step 1:** Compute $\hat{\mu}_k, \hat{\Sigma}, \hat{\pi}_k$ from the data (closed-form).
- **Step 2:** Compute $\mathbf{w}_{gen}$ and b_{gen} using the formulas above.
- **Step 3:** Initialize the Logistic Regression parameters $\theta_{init} = [\mathbf{w}_{gen}, b_{gen}]$.
- **Step 4:** Run Gradient Descent to fine-tune the weights.

This "Hybrid" approach often results in faster convergence and acts as a regularizer in low-data regimes.

Datasets

We will evaluate the Generative-Discriminative connection across three distinct datasets, ordered by complexity. This progression allows you to isolate implementation bugs (Dataset A) before analyzing convergence behavior (Dataset B) and regularization effects (Dataset C).

All datasets are pre-processed (standardized) and split into training and testing sets in the `data/` directory.

- **Dataset A: The Sanity Check (Iris)**
 - **Source:** A binary subset of Fisher's Iris dataset (Setosa vs. Versicolor).
 - **Dims:** $N = 100, D = 4$.
 - **Characteristics:** The classes are *linearly separable*.

- **Goal:** Both LDA and Logistic Regression should achieve nearly 100% accuracy. Use this to verify that your closed-form LDA implementation produces the correct decision boundary before tackling harder problems.

- **Dataset B: The Convergence Benchmark (Breast Cancer)**

- **Source:** Wisconsin Diagnostic Breast Cancer (WDBC).
- **Dims:** $N = 569, D = 30$.
- **Characteristics:** Features are computed from digitized images of cell nuclei (radius, texture, smoothness, etc.). The data is noisy but maintains a clear structure.
- **Goal:** This is the primary dataset for Task 3. You will observe that the random initialization takes significantly longer to converge compared to the LDA-initialized "Warm Start."

- **Dataset C: The High-Dimensional Challenge (Sonar)**

- **Source:** Connectionist Bench (Sonar, Mines vs. Rocks).
- **Dims:** $N = 208, D = 60$.
- **Characteristics:** Discriminates between sonar returns from a metal cylinder (Mines) and rocks.
- **Goal:** This dataset represents the "High Dimension, Low Sample Size" regime ($D \approx N/3$). Pure Logistic Regression is prone to overfitting here. You will analyze if the LDA prior acts as a regularizer, stabilizing the test performance.

Note: Feature scaling is critical for Gradient Descent. The provided data loader automatically standardizes the data ($\mu = 0, \sigma = 1$). Do not modify the preprocessing steps.

Task 1 The Closed-Form Generative Solution (LDA)

First, you must implement Linear Discriminant Analysis (LDA) from scratch. Unlike iterative approaches, LDA has a closed-form solution.

We assume the class-conditional densities are multivariate Gaussian with a **shared covariance matrix** Σ :

$$P(x|y = k) = \mathcal{N}(x|\mu_k, \Sigma)$$

1. Parameter Estimation:

Using the training data, estimate the following parameters using only `numpy`:

- **Class Priors** ($\hat{\pi}_k$): The fraction of samples belonging to class k .
- **Class Means** ($\hat{\mu}_k$): The mean vector for each class.
- **Pooled Covariance** ($\hat{\Sigma}$): Calculate the covariance for each class matrix, then compute the weighted average:

$$\hat{\Sigma} = \frac{N_0 \Sigma_0 + N_1 \Sigma_1}{N_0 + N_1}$$

2. Deriving Linear Weights:

The log-posterior ratio of LDA implies a linear decision boundary of the form $w^T x + b$. Derive and implement the function to convert LDA parameters into linear weights:

$$\mathbf{w}_{gen} = \hat{\Sigma}^{-1}(\hat{\mu}_1 - \hat{\mu}_0)$$

$$b_{gen} = -\frac{1}{2}\hat{\mu}_1^T \hat{\Sigma}^{-1} \hat{\mu}_1 + \frac{1}{2}\hat{\mu}_0^T \hat{\Sigma}^{-1} \hat{\mu}_0 + \ln\left(\frac{\hat{\pi}_1}{\hat{\pi}_0}\right)$$

Store these weights. You will use them to initialize the Logistic Regression model in Task 3.

Task 2 The Discriminative Optimizer

Next, you will implement a standard Logistic Regression model optimized via **Mini-Batch Gradient Descent**.

1. Model Definition:

Implement the sigmoid activation and the linear projection:

$$h_{\theta}(x) = \sigma(\mathbf{w}^T x + b) = \frac{1}{1 + e^{-(\mathbf{w}^T x + b)}}$$

2. Loss Function:

Implement the Binary Cross Entropy (Negative Log-Likelihood):

$$J(\mathbf{w}, b) = -\frac{1}{N} \sum_{i=1}^N \left[y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right]$$

3. Flexible Initialization:

Your fit method must accept an optional argument `init_params`.

- If `init_params` is `None`, initialize $\mathbf{w}$ and b using a standard normal distribution (Random Initialization).
- If `init_params` is provided, use those values as the starting point for Gradient Descent.

Task 3 The Hybrid Experiment (Convergence Analysis)

In this section, you will compare "Cold Start" (Random) vs. "Warm Start" (Generative) training.

1. Training Comparison:

Train two Logistic Regression models on the full training set for $T = 1000$ epochs with a fixed learning rate $\eta = 0.01$.

- **Model A (Random):** Initialize with $\mathcal{N}(0, 1)$.
- **Model B (Hybrid):** Initialize using $\mathbf{w}_{gen}$ and b_{gen} calculated in Task 1.

2. Analysis:

Plot the **Training Loss vs. Epochs** for both models on the same graph.

3. Observation:

You should observe that the Hybrid model starts at a significantly lower loss and converges faster. Report the initial loss value (J at epoch 0) for both strategies.

Important Instructions

1. You must use vectorized NumPy operations. Avoid explicit Python loops (e.g., `for i in range(N)`) when calculating predictions or gradients. Unvectorized code will time out.
2. **Numerical Stability:** When computing $\log(h(x))$, add a small epsilon ($\epsilon = 1e-9$) to avoid calculating $\log(0)$.
3. For Task 1, use `np.linalg.inv` or `np.linalg.pinv` (pseudo-inverse) if the covariance matrix is singular.
4. Please strictly follow the provided directory structure. Any deviation will result in a grade of 0.
5. All functions in `q1.py` are autograded. Ensure your code is free of syntax errors and runtime exceptions. A single error in a function may cause the autograder to fail for that specific task.
6. You can use `debug.py` for testing and debugging your code. You can print intermediate values or comment out sections to isolate issues.
7. You are only allowed to use NumPy. The import of any other library is not allowed.
8. Strictly adhere to the argument types, return types, and shapes specified in the docstrings. The autograder relies on exact matches for matrix dimensions (e.g., ensuring a return shape of $(N,)$ vs $(N, 1)$).

Q2. Decision Trees: The Geometry of Splitting

In previous assignments, you explored discriminative models like Logistic Regression (which creates linear decision boundaries) and generative models like GDA (which creates quadratic boundaries). However, many real-world datasets are not linearly or even quadratically separable.

Consider the **Two Moons** dataset. In a 2D feature space, one class forms an upper crescent while the other forms a lower crescent, interleaving each other. A simple linear classifier would fail to separate these classes without significant feature engineering.

In this lab, you will implement a **Classification Tree** from scratch. Unlike the previous models, the CART makes no strong assumptions about the data distribution. Instead, it recursively partitions the input space into hyper-rectangles, approximating complex non-linear decision boundaries (like the Moons) using a series of simple, axis-parallel splits.

The CART Framework

A Decision Tree classifies a data point x by passing it through a series of nodes. Each internal node represents a decision rule based on a single feature, and each leaf node represents a final predicted class.

Recursive Partitioning

Let D_i be the set of training examples $\{(x^{(n)}, y^{(n)})\}$ reaching node i . The goal is to split D_i into two purer subsets, D_L and D_R , based on a feature j and a threshold t .

The split is defined as:

$$\begin{aligned} D_L(j, t) &= \{(x, y) \in D_i \mid x_j \leq t\} \\ D_R(j, t) &= \{(x, y) \in D_i \mid x_j > t\} \end{aligned}$$

Model Fitting: The Greedy Approach

Finding the global optimal tree is NP-complete. Therefore, CART uses a **greedy** approach. At each node, it searches through every feature j and every unique threshold t to find the split that maximizes the "purity" of the resulting children.

To quantify purity, we use the **Gini Impurity**. For a node with data D , let $\hat{\pi}_c$ be the fraction of samples belonging to class c . The Gini Impurity is:

$$H(D) = 1 - \sum_{c=1}^K \hat{\pi}_c^2 \quad (2)$$

A pure node (all samples same class) has $H(D) = 0$.

The cost of a split (j, t) is the weighted average impurity of the children:

$$J(j, t) = \frac{|D_L|}{|D_i|} H(D_L) + \frac{|D_R|}{|D_i|} H(D_R) \quad (3)$$

The algorithm selects $(j^*, t^*) = \arg \min_{j, t} J(j, t)$.

Inference

To predict the class of a new input x_{new} :

1. Start at the root node.
2. If the current node is a leaf, return the majority class of the training samples in that leaf.
3. If the current node is a decision node with parameters (j, t) :
 - If $x_{new,j} \leq t$, traverse to the left child.
 - Else, traverse to the right child.

Task 1 Gini Impurity

The Gini Impurity measures the disorder of a set of labels. For a node with label array y , it is defined as:

$$\text{Gini}(y) = 1 - \sum_c p_c^2, \quad \text{where } p_c = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(y_i = c)$$

Implement the logic to compute this metric using vectorized NumPy operations. Ensure that your function handles the edge case where y is empty by returning 0.0.

Task 2 The Greedy Splitter

The core of CART is finding the optimal split (j^*, t^*) that minimizes the weighted impurity of the children. Implement the split-finding logic as follows:

1. **Search Space:** Iterate through every feature dimension j and every unique value in that column as a candidate threshold t .
2. **Partitioning:** For a candidate (j, t) , split the data into a left set $(x_j \leq t)$ and a right set $(x_j > t)$.
3. **Cost Calculation:** Compute the weighted Gini impurity:

$$J(j, t) = \frac{N_L}{N} \text{Gini}(y_L) + \frac{N_R}{N} \text{Gini}(y_R)$$

4. **Selection:** Track and return the parameters (j, t) that yield the minimum cost J . If a split results in an empty child node, it should be discarded.

For every candidate class, finding the optimal threshold can be computed efficient in $O(N)$ time where N is the number of data-points in the parent node. If we are searching t^* for the k th class, we first sort the dataset indices in the parent node as per the values of the k th feature. Now notice how does $J(j, t)$ change as you iterate over the sorted order of indices, we see that w.l.o.g. $|D_L|$ (left refers to smaller indices in the sorted order) always decreases and $|D_R|$ always increases in the same sense. Can we exploit our computation of $J(k, t)$ for every t we evaluate?

Task 3 Recursive Tree Growth

You are provided with a `Node` class structure in the skeleton code. Implement the recursive logic to build the tree:

1. **Stopping Criteria:** Before splitting, check if the recursion should terminate. Return a **Leaf Node** containing the majority class if:

- The maximum depth (`max_depth`) is reached.
 - The node is pure (all labels are identical).
 - The number of samples is below `min_samples_split`.
 - No valid split is found (e.g., all features are identical).
2. **Recursion:** If not stopping, find the best split using Task 2. Recursively call the growth function on the left and right subsets, and return a **Decision Node** storing the split parameters and references to the child nodes.
 3. **Inference:** Implement the traversal logic. To predict a class for input x , start at the root and move left ($x_j \leq t$) or right ($x_j > t$) until a Leaf Node is reached.

Task 4 The "Staircase" Analysis

The provided `main.py` script will train your tree on the non-linear **Two Moons** dataset and generate decision boundary plots. Run the script and analyze the outputs to answer the following:

1. **The Staircase Effect:** Compare the decision boundaries at `depth=3` and `depth=10`. Notice that CART approximates the smooth moon shape using axis-parallel rectangles. Does the deeper tree appear to capture the shape better, or is it memorizing noise?
2. **Bias-Variance Tradeoff:** Examine the training vs. test accuracy curve. At what depth does the model start to overfit? Explain this in terms of the complexity of the decision boundary.
3. **Geometric Instability:** Explain why Decision Trees are considered "high variance" estimators. If we rotated the dataset by 45° , would the tree structure (splits and thresholds) remain similar, or would it change drastically? Why?